ZEMING WU / RESEARCH PUBLICATIONS

Author-accepted manuscript

Published citation

Z. Wu, Z. Wang, and H. Zhang, “GPU-Accelerated Optimization-Based Collision Avoidance,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, Yokohama, Japan: IEEE, May 2024, pp. 7561–7567. DOI: 10.1109/ICRA57147.2024.10610871

This is the author’s accepted manuscript. Please cite the published IEEE article using the DOI above. The manuscript follows this notice page.

© 2024 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

GPU-Accelerated Optimization-Based Collision Avoidance

Zeming Wu[1], Zhuping Wang[1], and Hao Zhang[1]

***Abstract*— This paper proposes a GPU-accelerated optimization framework for collision avoidance problems where the controlled objects and the obstacles can be modeled as the finite union of convex polyhedra. A novel collision avoidance constraint is proposed based on scale-based collision detection and the strong duality of convex optimization. Under this constraint, the high-dimensional non-convex optimization problems of collision avoidance can be decomposed into several low-dimensional quadratic programmings (QPs) following the paradigm of alternating direction method of multipliers (ADMM). Furthermore, these low-dimensional QPs can be solved parallel with GPUs, significantly reducing computational time. High-fidelity simulations are conducted to validate the proposed method's effectiveness and practicality.**

I. Introduction

With the rapid advancement of autonomy, deploying autonomous systems in complex environments with obstacles has become increasingly prevalent. Examples include self-driving cars navigating on urban roads [1, 2] and autonomous quadrotors maneuvering through densely forested areas [3, 4]. In such scenarios, achieving high-precision collision avoidance is paramount to ensure safe navigation.

Various approaches have been developed to achieve collision avoidance, including search-based methods [5], sampling-based methods [6], and optimization-based methods [7–10]. Search-based methods discretize the configuration space into grids with a defined resolution and systematically search these grids to find a feasible solution. Sampling-based methods employ various sampling schemes to probe the configuration space for a feasible solution. While search-based and sampling-based methods have found extensive applications in the real world, they encounter computational challenges when precise consideration of both robot and obstacle geometry is necessary. This is notably evident in situations such as the intricate whole-body planning of quadrotors to navigate through narrow gaps [8] or overtaking in dense traffic flows [10].

Optimization-based methods, on the other hand, achieve safe navigation by minimizing a prescribed performance index and adhering to dynamics, kinematics, and collision avoidance constraints. Recent research [7–10] has demonstrated the advantages of optimization-based approaches, particularly in the context of high-precision collision avoidance. Furthermore, optimization-based methods, such as model predictive control (MPC) [7, 9] and trajectory optimization [8, 10] are inherently capable of finding better trajectories under certain performance index. Despite these promising features, there are three challenges hindering the development of optimization-based methods in real-world applications. Firstly, the precise formulation of collision avoidance constraint is hard to handle in general optimization frameworks. Secondly, the dimension of the optimization problem increases considerably with the number of obstacles. Thirdly, the optimization problems are generally non-convex due to the system dynamic constraints and collision avoidance constraints. These features make the optimization-based methods hard to achieve real-time navigation on embedded devices.

To address these challenges, this paper proposes a novel optimization-based framework for the collision avoidance. The geometry of controlled objectives and obstacles are modeled as finite unions of polyhedra, which can satisfy the precision requirements in most scenarios. Furthermore, the proposed framework can fully exploit the optimization problem's inherent structure, leveraging GPUs' power to expedite the solving process. The main contributions are summarized as follows.

- A novel scale-based collision avoidance constraint for polyhedra is proposed. Compared with the widely used signed distance-based constraint [7], the proposed constraint is linear with respect to dual variables.
- Leveraging the linear characteristic of the scale-based collision avoidance constraint, we break down the high-dimensional non-convex optimization problem of collision avoidance into multiple low-dimensional QPs following the paradigm of ADMM. These QPs can be solved parallel with GPUs, resulting in a significant reduction in computation time.
- High-fidelity simulations are conducted to evaluate the effectiveness and practicality of our framework on embedded platforms.

II. Related Works

In this section, a brief literature review of the optimization-based methods for collision avoidance is presented.

A. Collision Avoidance Constraints

The appropriate formulation of collision avoidance constraints is crucial for the computational overhead of optimization-based collision avoidance. In [11], collision avoidance constraints are established through the identification of a separating plane that distinguishes the quadrotor's position from obstacles on opposing sides. This approach leads to an optimization problem that involves integer variables, which takes several seconds to solve. In [12] the free space is decomposed into a series of convex overlapping polyhedra, namely safe flight corridors (SFCs), and then

[1]Z. Wu, Z. Wang and H. Zhang are with the Department of Control Science and Engineering, Tongji University, Shanghai, China, e-mail: {zemingwu, elewzp, zhang_hao}@tongji.edu.cn.

these polyhedra constraints are impose to optimization problem for generating collision-free trajectories. Nevertheless, SFCs can lead to conservative solutions due to the over-simplification of free space. For the applications requiring precise consideration of both robot and obstacle geometry, signed distance constraint [7, 8, 13] are commonly used. However, the signed distance constraint is implicit and non-differentiable, which makes it hard to handle in general optimization frameworks. Recently, a scale-based collision avoidance constraint has been proposed for convex set in [14]. Although this constraint is piecewise differentiable, it still retains an implicit and non-smooth nature. Moreover, calculating the gradient involves conic programming, which is computationally expensive on embedded platforms. To this end, an explicit, smooth and easily manageable constraint for collision avoidance is proposed in this paper.

B. Collision Avoidance with Optimization

Various approaches have been proposed to achieve collision avoidance with optimization. In works such as [8, 10, 15–17], the problem of collision-free trajectories generation is formulated as an unconstrained optimization problem. Within these frameworks, collision avoidance constraints are translated into penalty functions. However, these frameworks are bothered by unavoidable constraint violations, particularly in narrow, dense environments. Constrained optimization is introduced in [7, 9, 11, 13]. Unfortunately, it takes seconds to minutes to solve mixed-integer quadratic programming [11] or constrained nonlinear programming [7, 13], which makes them unsuitable for real-time navigation. As an attempt to enhance real-time performance, the high-dimensional non-convex optimization problem from [7] is converted to multiple low-dimensional second-order conic programmings (SOCPs) in [9], whose number is proportional to the number of obstacles and prediction horizon. Nevertheless, unlike QPs, there is no simplex or simplex-like solving method for SOCPs, making the method difficult to apply in practice. Worse, solving so many SOCPs with CPU remains computationally expensive on embedded platforms. In this paper, we tackle this challenges by separating the origin optimization problem into multiple low-dimensional QPs and leverage the power of GPU to expedite the solving process.

III. Scale Based Collision Avoidance Constraints of Polyhedra

In this section, the scale-based collision detection is introduced. Moreover, a novel scaled-based collision avoidance constraint is proposed using the strong duality of linear programming (LP).

A. Scale-based Collision Detection of Polyhedra

The geometry of obstacles and robots are both described by polyhedra. Specially, $A \in \mathbb{R}^{n_r \times 3}$ and $b \in \mathbb{R}^{n_r}$ are used to describe the robots:

$$A\boldsymbol{x} \preceq \boldsymbol{b}. \tag{1}$$

$C \in \mathbb{R}^{n_o \times 3}$ and $d \in \mathbb{R}^{n_o}$ are used to describe the obstacles:

$$C\boldsymbol{x} \preceq \boldsymbol{d}. \tag{2}$$

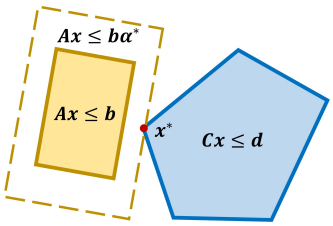


Fig. 1: An illustration of scale-based collision detection.

To detect whether there is collision between robot and obstacles, we need to check whether the polyhedra of robots and obstacles have overlap. One of the method to detect the overlap between polyhedra is proposed in [14]. This method enlarges/shrinks the polyhedra of both robot and obstacle to find a smallest scale $\alpha^* \in \mathbb{R}_+$ that cause a collision. If $\alpha^* < 1$, there is a collision between polyhedra. On the contrary, if $\alpha^* \geq 1$, there is no collision. In this paper, we only scale the polyhedra of robot, a geometry illustration is shown in Fig. 1. The collision detection problem between robot and obstacles is formulated as the following LP:

$$\begin{aligned} \min_{\alpha,\ \boldsymbol{x}} \quad & \alpha \\ s.t. \quad & A\boldsymbol{x} \preceq \boldsymbol{b}\ \alpha, \quad C\boldsymbol{x} \preceq \boldsymbol{d}. \end{aligned} \tag{3}$$

To enforce collision-free, we must impose the constraint $\alpha^* \geq 1$ into the optimization problem. This constraint is implicit and non-smooth, which is hard to handle in general optimization frameworks. Consequently, it is essential to reformulate this constraint into an explicit and smooth form.

B. Reformulation of Collision Constraints

We convert the collision avoidance constraint to an explicit one by using the strong duality of LP. The dual Lagrangian function of the LP (3) is:

$$\begin{aligned} g(\boldsymbol{\lambda}, \boldsymbol{\mu}) &= \inf_{\alpha,\boldsymbol{x}} \mathcal{L}(\alpha, \boldsymbol{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) \\ &= \inf_{\alpha,\boldsymbol{x}} \left\{ \alpha + \boldsymbol{\lambda}^T(A\boldsymbol{x} - \boldsymbol{b}\alpha) + \boldsymbol{\mu}^T(C\boldsymbol{x} - \boldsymbol{d}) \right\} \\ &= \inf_{\alpha} \left\{ (1 - \boldsymbol{\lambda}^T\boldsymbol{b})\alpha \right\} + \inf_{\boldsymbol{x}} \left\{ (\boldsymbol{\lambda}^T A + \boldsymbol{\mu}^T C)x \right\} - \boldsymbol{\mu}^T\boldsymbol{d}, \end{aligned} \tag{4}$$

where $\boldsymbol{\lambda}$ and $\boldsymbol{\mu}$ are the Lagrangian multiplier with proper dimension. Accordingly, the dual problem of LP (3) is:

$$\begin{aligned} \max_{\boldsymbol{\lambda},\ \boldsymbol{\mu}} \quad & -\boldsymbol{d}^T\boldsymbol{\mu} \\ s.t. \quad & -b^T\boldsymbol{\lambda} + 1 = 0, \quad \boldsymbol{\lambda} \succeq 0, \\ & A^T\boldsymbol{\lambda} + C^T\boldsymbol{\mu} = 0, \quad \boldsymbol{\mu} \succeq 0. \end{aligned} \tag{5}$$

Due to the strong duality of LP, the objective function of (5) and (3) have the following relationship:

$$-\boldsymbol{d}^T\boldsymbol{\mu} \leq -\boldsymbol{d}^T\boldsymbol{\mu}^* = \alpha^*. \tag{6}$$

Consequently, the collision-free constraint can be explicitly formulated as:

$$\begin{aligned} -\boldsymbol{d}^T\boldsymbol{\mu} \geq 1, \quad & \boldsymbol{\lambda} \succeq 0, \quad \boldsymbol{\mu} \succeq 0, \\ -b^T\boldsymbol{\lambda} + 1 = 0, \quad & A^T\boldsymbol{\lambda} + C^T\boldsymbol{\mu} = 0. \end{aligned} \tag{7}$$

It is worth pointing out that (7) are linear constraints with respect to $(\boldsymbol{\lambda},\ \boldsymbol{\mu})$. This feature makes it possible to decompose the optimization problem of collision avoidance into QPs.

IV. Collision Avoidance Based on Model Predictive Control

In this section, we employ the constraint (7) within the framework of MPC to formulate an optimization problem. Subsequently, in Section IV-B, the ADMM paradigm is adopted to decompose the high-dimensional non-convex optimization problem into multiple low-dimensional QPs. Further insights into the process of solving these QPs in parallel using GPUs are presented in Section IV-C. Finally, time complexity of the proposed method is discussed.

A. Optimization Problem Formulation

The robot state can be evaluated by the system equation:

$$\boldsymbol{s}_{t+1} = \boldsymbol{s}_t + f(\boldsymbol{s}_t, \boldsymbol{u}_t), \tag{8}$$

where $\boldsymbol{s}_t \in \mathbf{R}^{n_s}$ is the state of robot at t-th step, $\boldsymbol{u}_t \in \mathbb{R}^{n_u}$ is the control input at t-th step. Note that $f(\boldsymbol{s}_t, \boldsymbol{u}_t)$ is not a linear function in general.

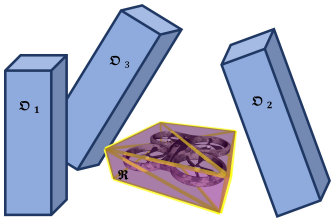


Fig. 2: The geometry of quadrotor and obstacles.

In this paper, the geometry of robot $\mathfrak{R}$ is modeled by the union of a series of compact convex set. Each compact convex set is modeled by a polyhedron that relates to the state of robot. More precisely:

$$\begin{aligned}\mathfrak{R} &= \mathfrak{R}_1 \cup \mathfrak{R}_2 \cup \cdots \cup \mathfrak{R}_N,\\ \mathfrak{R}_i &= \left\{\boldsymbol{y} \in \mathbb{R}^3 |\ \boldsymbol{y} = R(\boldsymbol{s}_t)\boldsymbol{x} + \boldsymbol{\rho}(\boldsymbol{s}_t),\ A_i\boldsymbol{x} \preceq \boldsymbol{b}_i\right\},\end{aligned}$$

where $R(\boldsymbol{s}_t) \in \mathbf{SO}(3)$ and $\boldsymbol{\rho}(\boldsymbol{s}_t) \in \mathbb{R}^3$ are the rotation matrix and translation vector, respectively. And $A_i \in \mathbb{R}^{n_{r,i}\times 3}$ and $\boldsymbol{b}_i \in \mathbb{R}^{n_{r,i}}$ are parameters need to be determined according to the shape and related location of the i-th part of robots. An example of quadrotor is shown in Fig. 2.

The geometry of obstacles is modeled by the union of a series of compact convex set, which is modeled by a polyhedron. That is:

$$\mathfrak{O} = \mathfrak{O}_1 \cup \mathfrak{O}_2 \cup \cdots \cup \mathfrak{O}_M, \tag{10a}$$

$$\mathfrak{O}_j = \left\{\boldsymbol{y} |\ C_j\boldsymbol{y} \preceq \boldsymbol{d}_j\right\}, \tag{10b}$$

where $C_j \in \mathbb{R}^{n_{o,j}\times 3}$ and $\boldsymbol{d}_j \in \mathbb{R}^{n_{o,j}}$ are parameters need to be determined according to the shape and location of the j-th obstacles. According to (7), the collision avoidance constraints between the i-th part of robot and the j-th obstacle at t-th step are obtained as:

$$\boldsymbol{\lambda}_{ijt} \succeq 0, \qquad \boldsymbol{\mu}_{ijt} \succeq 0, \qquad \gamma_{ijt} \geq 0, \tag{11a}$$

$$-\boldsymbol{b}_i^T\boldsymbol{\lambda}_{ijt} + 1 = 0, \tag{11b}$$

$$\mathcal{T}_{ijt}(\boldsymbol{s}_t, \boldsymbol{\mu}_{ijt}, \gamma_{ijt}) = 0, \tag{11c}$$

$$\mathcal{R}_{ijt}(\boldsymbol{s}_t, \boldsymbol{\lambda}_{ijt}, \boldsymbol{\mu}_{ijt}) = 0, \tag{11d}$$

where γ_{ijt} is the slack variable, and $\mathcal{T}_{ijt}$ is the nonlinear function relates to translation vector,

$$\begin{aligned}&\mathcal{T}_{ijt}(\boldsymbol{s}_t,\ \boldsymbol{\mu}_{ijt},\ \gamma_{ijt})\\ &\qquad = 1 + \left\{\boldsymbol{d}_j - C_j\boldsymbol{\rho}(\boldsymbol{s}_t)\right\}^T\boldsymbol{\mu}_{ijt} + \gamma_{ijt}.\end{aligned} \tag{12}$$

And $\mathcal{R}_{ijt}$ is the nonlinear function relates to rotation matrix,

$$\mathcal{R}_{ijt}(\boldsymbol{s}_t, \boldsymbol{\lambda}_{ijt}, \boldsymbol{\mu}_{ijt}) = A_i^T\boldsymbol{\lambda}_{ijt} + \left\{C_jR(\boldsymbol{s}_t)\right\}^T\boldsymbol{\mu}_{ijt}. \tag{13}$$

The MPC generates the control input $\boldsymbol{u}_t$ by forecasting T-step future states and optimizing a cost function $\mathcal{F}$ under system, boundary and collision avoidance constraints. The explicit form of cost function $\mathcal{F}$ depends on the requirements of task. For instance, in trajectory tracking tasks, the tracking error and control efforts are take into account:

$$\mathcal{F}(\{\boldsymbol{s}_t, \boldsymbol{u}_t\}) = \sum_{t=0}^{T}\left(\|\boldsymbol{s}_t - \tilde{\boldsymbol{s}}_t\|_{Q_s}^2 + \|\boldsymbol{u}_t\|_{Q_u}^2\right), \tag{14}$$

where $\tilde{\boldsymbol{s}}_t$ is the reference state, Q_s and Q_u are positive-definite matrixes with proper dimension. The optimization problem of MPC is finally formulated as follows:

$$\min_{\substack{\{\boldsymbol{s}_t,\boldsymbol{u}_t\},\\ \{\boldsymbol{\lambda}_{ijt},\boldsymbol{\mu}_{ijt},\gamma_{ijt}\}.}} \quad \sum_{t=0}^{T}\left(\|\boldsymbol{s}_t - \tilde{\boldsymbol{s}}_t\|_{Q_s}^2 + \|\boldsymbol{u}_t\|_{Q_u}^2\right) \tag{15a}$$

$$s.t. \quad \boldsymbol{s}_{t+1} = \boldsymbol{s}_t + f(\boldsymbol{s}_t, \boldsymbol{u}_t),\ \forall t, \tag{15b}$$

$$\boldsymbol{s}_{min} \preceq \boldsymbol{s}_t \preceq \boldsymbol{s}_{max},\ \forall t, \tag{15c}$$

$$\boldsymbol{u}_{min} \preceq \boldsymbol{u}_t \preceq \boldsymbol{u}_{max},\ \forall t, \tag{15d}$$

$$\boldsymbol{\lambda}_{ijt} \succeq 0,\quad \boldsymbol{\mu}_{ijt} \succeq 0,\quad \gamma_{ijt} \geq 0,\ \forall i,j,t, \tag{15e}$$

$$-\boldsymbol{b}_i^T\boldsymbol{\lambda}_{ijt} + 1 = 0,\ \forall i,j,t, \tag{15f}$$

$$\mathcal{T}_{ijt}(\boldsymbol{s}_t, \boldsymbol{\mu}_{ijt}, \gamma_{ijt}) = 0,\ \forall i,j,t, \tag{15g}$$

$$\mathcal{R}_{ijt}(\boldsymbol{s}_t, \boldsymbol{\lambda}_{ijt}, \boldsymbol{\mu}_{ijt}) = 0,\ \forall i,j,t, \tag{15h}$$

where (15b) is the system dynamic constraints, (15c)-(15d) are the constraints on states and control input, and (15e)-(15h) are the collision avoidance constraints.

There are two challenges to solving the optimization problem (15). Firstly, the dimension of the optimization problem is related to the number of obstacles M. When M increases, the dimension of the optimization problem increases considerably, and hence real-time performance can not be guaranteed. Secondly, the constraints (15b), (15g) and (15h) are non-convex constraints in general, which makes the optimization problem hard to solve.

In this paper, we circumvent the first challenge by decomposing the original high-dimensional problem into several low-dimensional problems that can be solved in parallel. Noting that for different (i,j,t), the variables $(\boldsymbol{\lambda}_{ijt},\ \boldsymbol{\mu}_{ijt},\ \gamma_{ijt})$ are coupled indirectly with each other. This property implies the potential to decompose optimization problem (15) into several sub-problems only with $(\boldsymbol{\lambda}_{ijt},\ \boldsymbol{\mu}_{ijt},\ \gamma_{ijt})$ as variables. To this end, the ADMM [18] is used to decompose the original problem (15), which is detailed in in Subsection IV-B. We handle the second challenge by linearizing $f(\boldsymbol{s}_t, \boldsymbol{u}_t)$, $R(\boldsymbol{s}_t)$ and $\boldsymbol{\rho}(\boldsymbol{s}_t)$ with respect to $(\boldsymbol{s}_t,\ \boldsymbol{u}_t)$.

B. Problem Separation via ADMM

The ADMM handles the coupled constraints with augmented Lagrangian, and the augmented Lagrangian of (15) is defined as:

$$\begin{aligned}
&\mathcal{L}_\sigma(\{\boldsymbol{s}_t,\ \boldsymbol{u}_t\},\ \{\boldsymbol{\lambda}_{ijt},\ \boldsymbol{\mu}_{ijt}, \gamma_{ijt}\},\ \{\zeta_{ijt}, \boldsymbol{\xi}_{ijt}\}) \\
&= \quad \mathcal{F}(\{\boldsymbol{s}_t, \boldsymbol{u}_t\}) \quad + \quad IC_0(\{\boldsymbol{s}_t, \boldsymbol{u}_t\}) + \\
&\frac{\sigma}{2} \sum_{i=1}^{N} \sum_{j=1}^{M} \sum_{t=0}^{T} \left\| \mathcal{T}_{ijt}(\boldsymbol{s}_t,\ \boldsymbol{\mu}_{ijt},\ \gamma_{ijt}) + \zeta_{ijt} \right\|_2^2 + \\
&\frac{\sigma}{2} \sum_{i=1}^{N} \sum_{j=1}^{M} \sum_{t=0}^{T} \left\| \mathcal{R}_{ijt}(\boldsymbol{s}_t,\ \boldsymbol{\lambda}_{ijt},\ \boldsymbol{\mu}_{ijt}) + \boldsymbol{\xi}_{ijt} \right\|_2^2 + \\
&\sum_{i=1}^{N} \sum_{j=1}^{M} \sum_{t=0}^{T} IC_{ijt}(\boldsymbol{\lambda}_{ijt},\ \boldsymbol{\mu}_{ijt},\ \gamma_{ijt}),
\end{aligned} \tag{16}$$

where $\sigma > 0$ is the penalty parameter. ζ_{ijt} and $\boldsymbol{\xi}_{ijt}$ are the dual variables of ADMM. $IC_0(\{\boldsymbol{s}_t, \boldsymbol{u}_t\})$ is the indicator function of constraints (15b) - (15d). That is, $IC_0(\{\boldsymbol{s}_t, \boldsymbol{u}_t\}) = 0$ if constraints (15b) - (15d) are satisfied, and $IC_0(\{\boldsymbol{s}_t, \boldsymbol{u}_t\}) = \infty$ otherwise. $IC_{ijt}(\boldsymbol{\lambda}_{ijt},\ \boldsymbol{\mu}_{ijt},\ \gamma_{ijt})$ is the indicator function of constraints (15e) - (15f).

The ADMM is an iterative algorithm to solve optimization problems. In each iteration, the ADMM updates the optimization variables via solving sub-problems. Specially, for augmented Lagrangian (16), there are three steps. In $(k+1)$-th iteration, the first step is the updates of dual variables:

$$\begin{aligned}
&\{\boldsymbol{\lambda}_{ijt}^{k+1}, \boldsymbol{\mu}_{ijt}^{k+1}, \gamma_{ijt}^{k+1}\} = \\
&\underset{\{\boldsymbol{\lambda}_{ijt}, \boldsymbol{\mu}_{ijt}, \gamma_{ijt}\}}{\arg\min}\ \mathcal{L}_\sigma(\{\boldsymbol{s}_t^k, \boldsymbol{u}_t^k\}, \{\boldsymbol{\lambda}_{ijt}, \boldsymbol{\mu}_{ijt}, \gamma_{ijt}\}, \{\zeta_{ijt}^k, \boldsymbol{\xi}_{ijt}^k\}).
\end{aligned} \tag{17}$$

The second step is the updates of states and control inputs:

$$\begin{aligned}
&\{\boldsymbol{s}_t^{k+1}, \boldsymbol{u}_t^{k+1}\} = \\
&\underset{\{\boldsymbol{s}_t, \boldsymbol{u}_t\}}{\arg\min}\ \mathcal{L}_\sigma(\{\boldsymbol{s}_t, \boldsymbol{u}_t\}, \{\boldsymbol{\lambda}_{ijt}^k, \boldsymbol{\mu}_{ijt}^k, \gamma_{ijt}^k\}, \{\zeta_{ijt}^k, \boldsymbol{\xi}_{ijt}^k\}).
\end{aligned} \tag{18}$$

The third step is the updates of dual variables of ADMM:

$$\begin{aligned}
\zeta_{ijt}^{k+1} &= \zeta_{ijt}^k + \mathcal{T}_{ijt}(\boldsymbol{s}_t^k,\ \boldsymbol{\mu}_{ijt}^k,\ \gamma_{ijt}^k),\ \forall i, j, t, \\
\boldsymbol{\xi}_{ijt}^{k+1} &= \boldsymbol{\xi}_{ijt}^k + \mathcal{R}_{ijt}(\boldsymbol{s}_t^k,\ \boldsymbol{\lambda}_{ijt}^k,\ \boldsymbol{\mu}_{ijt}^k),\ \forall i, j, t.
\end{aligned} \tag{19}$$

The stopping criteria terminating the iterative procedure is given by the following conditions,

$$\sum_i \sum_j \sum_t \left(\|\zeta_{ijt}^{k+1} - \zeta_{ijt}^k\|_2^2 + \|\boldsymbol{\xi}_{ijt}^{k+1} - \boldsymbol{\xi}_{ijt}^k\|_2^2 \right) \leq \epsilon_{pri}, \tag{20}$$

$$\sum_i \sum_j \sum_t \left(\|\boldsymbol{\lambda}_{ijt}^{k+1} - \boldsymbol{\lambda}_{ijt}^k\|_2^2 + \|\boldsymbol{\mu}_{ijt}^{k+1} - \boldsymbol{\mu}_{ijt}^k\|_2^2 \right) \leq \epsilon_{dual}, \tag{21}$$

where the $\epsilon_{pri} > 0$ and $\epsilon_{dual} > 0$ are the constants that control the feasibility of constants (15g)-(15h) and optimality of solution, respectively. To trade off between feasibility and optimality, ϵ_{pri} and ϵ_{dual} need to be chosen empirically. The overview of our optimization process is shown in Fig. 3.

Noting that the variables with superscript are regarded as constant value in (17)-(19). Consequently, the optimization problem (17) is a QP with linear constraints. Furthermore, optimization problem (17) is inherently decoupled

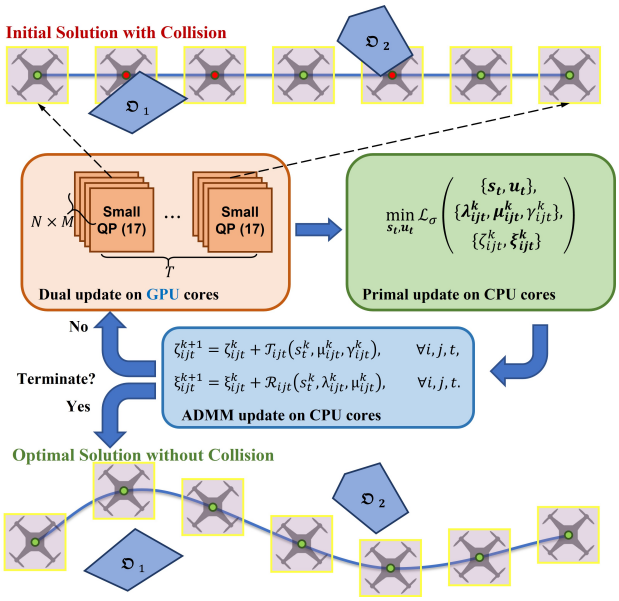


Fig. 3: The overview of our optimization process.

for variables with different (i, j, t). Consequently, optimization problem (17) can be separated into a series of low-dimensional QPs that can be solved in parallel with GPUs. We will discuss about the parallel solving method in IV-C.

The second challenge mentioned before appears in the optimization problem (18). That is, the optimization problem (18) is a non-linear programming (NLP) due to the nonlinearity of $f(\boldsymbol{s}_t, \boldsymbol{u}_t)$, $R(\boldsymbol{s}_t)$ and $\boldsymbol{\rho}(\boldsymbol{s}_t)$. In this paper, the NLP is optimized with sequential quadratic programming (SQP). More precisely, the NLP is approximated by a series of QPs. To enhance real-time performance, we only solve the QP once in each iteration of ADMM.

C. Solving Multiple QPs with GPU in Parallel

The sub-problems in (17) are equivalent to the following QP,

$$\min_{\boldsymbol{y}_{ijt}} \quad \frac{1}{2} \|K_{ijt}^T\ \boldsymbol{y}_{ijt} + \boldsymbol{b}_{ijt}\|_2^2 \tag{22a}$$

$$s.t. \quad \boldsymbol{\kappa}_{ijt}^T\ \boldsymbol{y}_{ijt} = \boldsymbol{\eta}_{ijt}, \tag{22b}$$

$$\boldsymbol{y}_{ijt} \succeq \mathbf{0}, \tag{22c}$$

where the parameters are detailed as,

$$\boldsymbol{y}_{ijt} = \begin{bmatrix} \boldsymbol{\lambda}_{ijt}^T & \boldsymbol{\mu}_{ijt}^T & \gamma_{ijt} \end{bmatrix}^T \in \mathbb{R}^{n_r + n_o + 1}, \tag{23a}$$

$$K_{ijt} = \begin{pmatrix} \mathbf{0} & A_i \\ \boldsymbol{d}_j - C_j \boldsymbol{\rho}_t & C_j R_t \\ 1 & \mathbf{0} \end{pmatrix} \in \mathbb{R}^{(n_r + n_o + 1) \times 4}, \tag{23b}$$

$$\boldsymbol{b}_{ijt} = \begin{pmatrix} 1 + \zeta_{ijt} \\ \boldsymbol{\xi}_{ijt} \end{pmatrix} \in \mathbb{R}^4, \tag{23c}$$

$$\boldsymbol{\kappa}_{ijt} = \begin{pmatrix} \boldsymbol{b}_i \\ \mathbf{0} \end{pmatrix} \in \mathbb{R}^{n_r + n_o + 1}, \quad \boldsymbol{\eta}_{ijt} = 1. \tag{23d}$$

There are $(N \times M \times T)$ QPs in total, which is the bottleneck of the ADMM updates. We avoid this bottleneck by solving these QPs in parallel. More precisely, the parallel

version of Lemke's algorithm [19] are implemented on GPU and used to solve these problem. To use Lemke's algorithm, the QPs (22) should be converted into linear complementarity problems (LCPs). The conversion are separated in two steps.

For simplicity, the subscripts (i, j, t) are omitted in the remainder of this subsection. The first step is to convert the linear equality constraint (22b) into linear inequality constraint. The variables and parameters are partitioned as:

$$\boldsymbol{y} = \begin{pmatrix} \boldsymbol{y}_e \\ \boldsymbol{y}_u \end{pmatrix}, \ K = \begin{pmatrix} K_e \\ K_u \end{pmatrix}, \ \boldsymbol{\kappa} = \begin{pmatrix} \boldsymbol{\kappa}_e \\ \boldsymbol{\kappa}_u \end{pmatrix}, \tag{24}$$

where $\boldsymbol{\kappa}_e$ is an invertible square matrix with proper dimension. According to (22b), we have:

$$\boldsymbol{y}_e = \boldsymbol{\kappa}_e^{-T}(\boldsymbol{\eta} - \boldsymbol{\kappa}_u^T \boldsymbol{y}_u). \tag{25}$$

Submitting the above equation into (22), we have the following new QP,

$$\min_{\boldsymbol{y}_u} \quad \frac{1}{2}\|\tilde{K}^T \boldsymbol{y}_u + \tilde{\boldsymbol{b}}\|_2^2 \tag{26a}$$

$$s.t. \quad \tilde{\boldsymbol{\kappa}}^T \ \boldsymbol{y}_u \preceq \tilde{\boldsymbol{\eta}}, \tag{26b}$$

$$\boldsymbol{y}_u \succeq \boldsymbol{0}, \tag{26c}$$

where the new parameters are detailed as,

$$\tilde{K} = K_u - \boldsymbol{\kappa}_u \boldsymbol{\kappa}_e^{-1} K_e, \quad \tilde{\boldsymbol{b}} = \boldsymbol{b} + (\boldsymbol{\kappa}_e^{-1} K_e)^T \boldsymbol{\eta}, \tag{27a}$$

$$\tilde{\boldsymbol{\kappa}} = \boldsymbol{\kappa}_u \boldsymbol{\kappa}_e^{-1}, \quad \tilde{\boldsymbol{\eta}} = \boldsymbol{\kappa}_e^{-T} \boldsymbol{\eta}. \tag{27b}$$

The second step is to convert the new QP (26) into LCP. The Karush-Kuhn-Tucker (KKT) conditions of (26) is,

$$\tilde{K}\tilde{K}^T \boldsymbol{y}_u + \tilde{K}\tilde{\boldsymbol{b}} + \tilde{\boldsymbol{\kappa}}\boldsymbol{\phi} - \boldsymbol{\psi} = \boldsymbol{0}, \tag{28a}$$

$$\tilde{\boldsymbol{\kappa}}^T \boldsymbol{y}_u - \tilde{\boldsymbol{\eta}} + \boldsymbol{\varepsilon} = \boldsymbol{0}, \tag{28b}$$

$$\boldsymbol{\phi}^T \boldsymbol{\varepsilon} = 0, \quad \boldsymbol{\psi}^T \boldsymbol{y}_u = 0, \tag{28c}$$

$$\boldsymbol{\phi} \succeq \boldsymbol{0}, \quad \boldsymbol{\varepsilon} \succeq \boldsymbol{0}, \quad \boldsymbol{y}_u \succeq \boldsymbol{0}, \quad \boldsymbol{\psi} \succeq \boldsymbol{0}, \tag{28d}$$

where $\boldsymbol{\phi}$ and $\boldsymbol{\psi}$ are the dual variables of inequality constraints (26b) and (26c), respectively. And $\boldsymbol{\varepsilon}$ is the slack variable of (26b). The KKT conditions (28) is equivalent to the following LCP:

$$\begin{aligned} \boldsymbol{w} = M\boldsymbol{z} + \boldsymbol{q}, \quad \boldsymbol{w}^T \boldsymbol{z} = 0, \\ \boldsymbol{w} \succeq \boldsymbol{0}, \quad \boldsymbol{z} \succeq \boldsymbol{0}, \end{aligned} \tag{29}$$

where the variables and parameters are detailed as,

$$\begin{aligned} \boldsymbol{z} = \begin{pmatrix} \boldsymbol{y_u} \\ \boldsymbol{\phi} \end{pmatrix}, \quad \boldsymbol{w} = \begin{pmatrix} \boldsymbol{\psi} \\ \boldsymbol{\varepsilon} \end{pmatrix}, \\ M = \begin{pmatrix} \tilde{K}\tilde{K}^T & \tilde{\kappa} \\ -\tilde{\kappa}^T & \boldsymbol{0} \end{pmatrix}, \quad \boldsymbol{q} = \begin{pmatrix} \tilde{K}\tilde{\boldsymbol{b}} \\ \tilde{\eta} \end{pmatrix}. \end{aligned} \tag{30}$$

The conversion is completed.

D. Computational Complexity

There are two levels of acceleration to solve (17) with GPU. The first level is the acceleration of solving a single sub-problem. Denotes n as the dimension of $\boldsymbol{z}$, the time complexity of solving QP with single thread CPU program is $O(n^3)$. On the contrary, the Lemke's algorithm typically requires $O(n)$ iterations to terminate [20], and the time complexity of each iteration is $O(n)$ when implemented with GPU [19]. As a result, the time complexity for solving one sub-problem with GPU is of order $O(n^2)$.

The second level is the acceleration of solving multiple sub-problems. There are $(N \times M \times T)$ sub-problems in total. These sub-problems can be solved simultaneously using multi-threads technique of both CPU and GPU. A CPU generally has 4-16 threads, on the contrary, a GPU can generally handle thousands of threads within one clock cycle. Hence, GPUs outperform CPUs when the number of sub-problems is large. For CPU, the time complexity for solving all sub-problems of (17) is of order,

$$O\left(ceil(\frac{NMT}{K_{CPU}}) \cdot (n_{max}+1)^3\right), \tag{31}$$

where $ceil(x)$ is the function that computes the smallest integer that is greater than or equal to x. K_{CPU} is the number of threads of CPU, and $n_{max} = \max_{i,j} (n_{r,i} + n_{o,j})$ is the maximum dimension of sub-problems. For GPU, the time complexity for solving all sub-problems of (17) is of order:

$$O\left(ceil(\frac{NMT}{ceil(K_{GPU}/(n_{max}+1))}) \cdot (n_{max}+1)^2\right), \tag{32}$$

where K_{GPU} is the number of threads of GPU. It is worth mentioning that $K_{GPU} \gg K_{CPU}$ in general.

V. Simulation and Benchmark Comparisons

In this section, high-fidelity simulations on quadrotor are conducted to illustrate the effectiveness of our algorithm. We focus on the collision-free trajectory tracking problem of quadrotors. More precisely, the quadrotor should move as close as possible to a reference trajectory while avoiding collisions with obstacles.

The proposed method is also compared with the following state-of-the-art optimization based method.

1) Optimization-based collision avoidance (OBCA) [7], which use sign distance-based collision avoidance constraints and solve the MPC problem via non-convex optimization method like quasi-newton method or sequential convex programming. The OBCA method is generally regarded as an offline planner.
2) Regularized dual alternating direction method of multiplier (RDA) [9], which use sign distance-based collision avoidance constraints. RDA formulate the update of dual variables as SOCPs. And these SOCPs are solved using multi-threads technique of CPU.

All these method are evaluated on an embedded platform called Nvidia Jetson Xavier NX.

A. High-Fidelity Simulation

A high-fidelity quadrotor simulator called AirSim [21] is used to verify the effectiveness of the proposed method under practical sensor and dynamic constraints. The simulator is running on a laptop equipped with an Intel Core i9-12700 CPU and a GeForce RTX 3070 Ti GPU. The simulation scenario is of size $70m \times 20m \times 6m$ and contains 32 obstacles

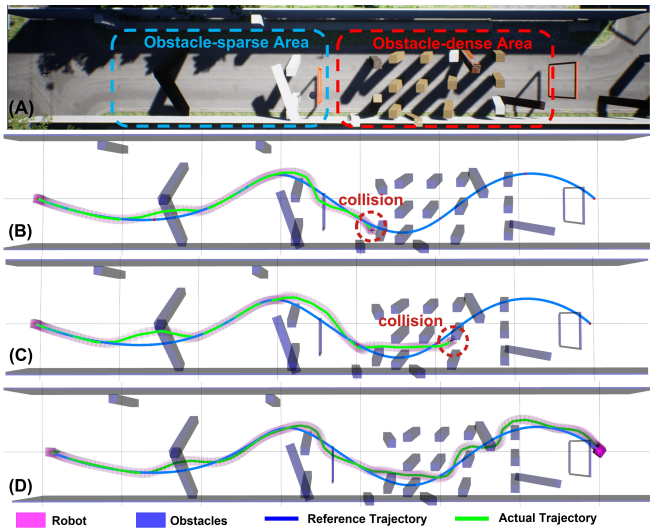


Fig. 4: One case in the simulation. (A) The environment in AirSim. (B) The geometry of obstacles, the geometry of the quadrotor, the reference trajectory and the actual trajectory during the navigation with OBCA. (C) The navigation with RDA. (D) The navigation with the proposed method.

in total. In particular, the scenario contains an obstacle-sparse area and an obstacle-dense area as shown in Fig. 4(A).

For all method, the discrete time $\Delta t = 0.1s$, the prediction horizon $T = 16$. The penalty parameter σ of RDA and proposed method is set to 300. The initial position of quadrotor is $(0, 0, 1)$, the goal position is $(0, 70, 1)$, the duration of reference trajectories is set to $25s$. The reference trajectories are generated by interpolating initial position, goal position and 5 random waypoints. During the navigation, the quadrotor can only sense the obstacles within $20m \times 20m \times 6m$.

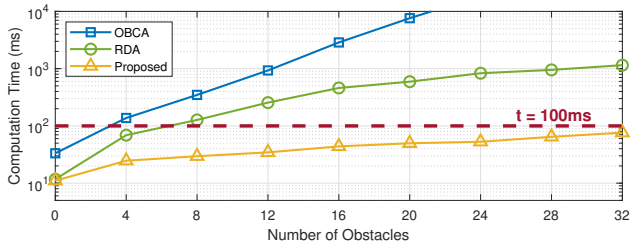
One particular case of different methods is shown in Fig. 4(B)-(C). The quadrotor successfully passes through the obstacle-sparse area with all three methods. However, the OBCA and RDA methods lead to a collision in the obstacle-dense area. Table I reports the success rate, navigation times, and navigation cost $\mathcal{F}$ (14) of 100 trials. The three methods achieve similar performance in navigation time and navigation cost, while the proposed method outperforms OBCA and RDA in success rate.

TABLE I: BENCHMARK COMPARISON OF SIMULATIONS

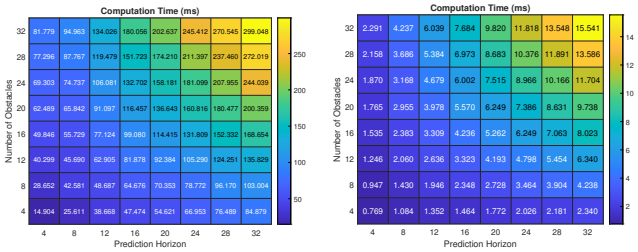
Method	success rate	Navigation Time*			Navigation Cost* $\mathcal{F}$		
		min	max	mean	min	max	mean
OBCA [7]	35%	**26.6s**	32.7s	**28.3s**	380.7	**532.6**	483.9
RDA [9]	58%	27.2s	**32.1s**	28.5s	348.1	576.1	492.6
Proposed	**92%**	26.8s	33.8s	28.7s	**322.4**	558.2	**458.7**

* Only the data of success trials are taken into account.

To further explain the result in Table I, the computational time of different methods is summarized in Fig. 5. The OBCA method solve the optimization problem (15) directly, whose dimension is proportional to the number of obstacles M. Consequently, the computational time of OBCA method increase dramatically with M. As shown in Fig. 5(a), the



(a) Average computation time of one control loop with different methods when prediction horizon $T = 16$.



(b) Dual updates of RDA. (c) Dual updates of our method.

Fig. 5: Comparisons of computation time.

OBCA method can not generate safe trajectories within $0.1s$ in the obstacle-dense area, which causes collisions. The RDA method and the proposed method both solve the optimization problem (15) with ADMM. The main difference between the two methods is that the RDA method uses distance-based collision detection, which leads to solving a series of SOCPs to update dual variables. These SOCPs are solved in parallel via the thread pool technique of CPUs. On the contrary, the proposed method uses scale-based collision detection, which leads to solving a series of QPs to update dual variables as shown in Section IV-B. Moreover, these QPs are solved in parallel via GPUs. We compare the computation times of dual updates between the RDA method and the proposed method on an embedded platform called Xavier NX, which has 6 CPU cores and 384 GPU cores. The results are shown in Fig. 5(b) and Fig. 5(c). It can be seen that the proposed method outperforms the RDA method by orders of magnitudes. As a result, the proposed method can still generate safe trajectories timely in obstacle-dense area.

VI. CONCLUSION

This paper proposed a GPU-accelerated optimization framework for the collision avoidance problem. With the help of scale-based collision detection and ADMM, the optimization problem is separated into multiple low-dimensional QPs, which can be solved in parallel with GPUs. High-fidelity simulations on quadrotors are conducted to show the effectiveness of the proposed framework. The simulation results have shown that the proposed framework significantly reduces the computational time of optimization problems. Moreover, the benchmark comparisons indicate that the proposed framework performs better on embedded platforms than OBCA and RDA.

REFERENCES

[1] Z. Wang, G. Li, H. Jiang, Q. Chen, and H. Zhang, "Collision-free navigation of autonomous vehicles using convex quadratic programming-based model predictive control," *IEEE/ASME Trans. Mechatronics*, vol. 23, no. 3, pp. 1103–1113, 2018.

[2] W. Ding, L. Zhang, J. Chen, and S. Shen, "Epsilon: An efficient planning system for automated vehicles in highly interactive environments," *IEEE Trans. Robot.*, vol. 38, no. 2, pp. 1118–1138, 2021.

[3] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Learning high-speed flight in the wild," *Sci. Robot.*, vol. 6, no. 59, p. eabg5810, 2021.

[4] X. Zhou, X. Wen, Z. Wang, Y. Gao, H. Li, Q. Wang, T. Yang, H. Lu, Y. Cao, C. Xu *et al.*, "Swarm of micro flying robots in the wild," *Sci. Robot.*, vol. 7, no. 66, p. eabm5954, 2022.

[5] S. Liu, K. Mohta, N. Atanasov, and V. Kumar, "Search-based motion planning for aggressive flight in $\mathrm{SE}(3)$," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 2439–2446, 2018.

[6] Z. Wang, Y. Li, H. Zhang, C. Liu, and Q. Chen, "Sampling-based optimal motion planning with smart exploration and exploitation," *IEEE/ASME Trans. Mechatronics*, vol. 25, no. 5, pp. 2376–2386, 2020.

[7] X. Zhang, A. Liniger, and F. Borrelli, "Optimization-based collision avoidance," *IEEE Trans. Control Syst. Technol.*, vol. 29, no. 3, pp. 972–983, 2020.

[8] Z. Wang, X. Zhou, C. Xu, and F. Gao, "Geometrically constrained trajectory optimization for multicopters," *IEEE Trans. Robot.*, vol. 38, no. 5, pp. 3259–3278, 2022.

[9] R. Han, S. Wang, S. Wang, Z. Zhang, Q. Zhang, Y. C. Eldar, Q. Hao, and J. Pan, "RDA: An accelerated collision free motion planner for autonomous navigation in cluttered environments," *IEEE Robot. Autom. Lett.*, vol. 8, no. 3, pp. 1715–1722, 2023.

[10] Q. Wang, Z. Wang, L. Pei, C. Xu, and F. Gao, "A linear and exact algorithm for whole-body collision evaluation via scale optimization," in *Proc. IEEE Int. Conf. Robot. Autom.* IEEE, 2023, pp. 3621–3627.

[11] D. Mellinger, A. Kushleyev, and V. Kumar, "Mixed-integer quadratic program trajectory generation for heterogeneous quadrotor teams," in *Proc. IEEE Int. Conf. Robot. Autom.* IEEE, 2012, pp. 477–483.

[12] S. Liu, M. Watterson, K. Mohta, K. Sun, S. Bhattacharya, C. J. Taylor, and V. Kumar, "Planning dynamically feasible trajectories for quadrotors using safe flight corridors in 3-d complex environments," *IEEE Robot. Autom. Lett.*, vol. 2, no. 3, pp. 1688–1695, 2017.

[13] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, S. Patil, K. Goldberg, and P. Abbeel, "Motion planning with sequential convex optimization and convex collision checking," *Int. J. Robot. Res.*, vol. 33, no. 9, pp. 1251–1270, 2014.

[14] K. Tracy, T. A. Howell, and Z. Manchester, "Differentiable collision detection for a set of convex primitives," in *Proc. IEEE Int. Conf. Robot. Autom.* IEEE, 2023, pp. 3663–3670.

[15] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "Stomp: Stochastic trajectory optimization for motion planning," in *Proc. IEEE Int. Conf. Robot. Autom.* IEEE, 2011, pp. 4569–4574.

[16] M. Zucker, N. Ratliff, A. D. Dragan, M. Pivtoraiko, M. Klingensmith, C. M. Dellin, J. A. Bagnell, and S. S. Srinivasa, "Chomp: Covariant hamiltonian optimization for motion planning," *Int. J. Robot. Res.*, vol. 32, no. 9-10, pp. 1164–1193, 2013.

[17] Z. Han, Z. Wang, N. Pan, Y. Lin, C. Xu, and F. Gao, "Fast-racing: An open-source strong baseline for $\mathrm{SE}(3)$ planning in autonomous drone racing," *IEEE Robot. Autom. Lett.*, vol. 6, no. 4, pp. 8631–8638, 2021.

[18] S. Boyd, N. Parikh, E. Chu, B. Peleato, J. Eckstein *et al.*, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Foundations and Trends® in Machine learning*, vol. 3, no. 1, pp. 1–122, 2011.

[19] P. Kipfer, "Lcp algorithms for collision detection using cuda," *GPU Gems*, vol. 3, pp. 723–740, 2007.

[20] J. E. Lloyd, "Fast implementation of lemke's algorithm for rigid body contact simulation," in *Proc. IEEE Int. Conf. Robot. Autom.* IEEE, 2005, pp. 4538–4543.

[21] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "Airsim: High-fidelity visual and physical simulation for autonomous vehicles," in *Proc. Field Serv. Robot. Int. Conf.* Springer, 2018, pp. 621–635.